Maze Solving

Maze Runner

Author

O. Denis

Abstract

This project compares two algorithmic approaches for solving randomly generated mazes: an approach based on an optimized Dijkstra algorithm and an approach using a genetic algorithm with pheromones. The three components of the system (DFS generation, Dijkstra solving, genetic optimization) are implemented in Python. Empirical analysis confirms $\mathcal{O}(n^2)$ complexity for both solvers on $n \times n$ mazes, with quality and speed favoring Dijkstra for optimal solutions, while the GA explores the search space in a more creative manner.

Contents

1 Specification and Design **2**
1.1 General Architecture . . . 2
1.2 Maze Generation (DFS) . . . 2
1.3 Solving by Dijkstra (BFS) . . . 3
1.4 Genetic Algorithm with Pheromones . . . 3

2 Code Organization **4**
2.1 Architecture . . . 4
2.2 Control Flow . . . 5
2.3 Memory Management . . . 5

3 Installation and Usage **6**
3.1 Requirements . . . 6
3.2 Running . . . 6
3.3 Configurable Parameters . . . 6

4 Behavioral Analysis **6**
4.1 Empirical Complexity . . . 6
4.2 Fitness Convergence (GA) . . . 7
4.3 Path Lengths . . . 7
4.4 GA Parameter Sensitivity . . . 8
4.5 Exploration vs. Exploitation of Pheromones . . . 8
4.6 Dijkstra vs. GA Comparison . . . 9
4.7 Connectivity . . . 9

5 Formal Proofs **9**
5.1 GA Convergence . . . 10

6 Implemented Optimizations **10**

7 Experimental Results **11**
7.1 Mazes Tested . . . 11
7.2 Dijkstra Solutions . . . 11
7.3 GA Solutions (1000 generations) . . . 11

8 Conclusion **11**
8.1 Key Results . . . 11
8.2 Possible Improvements . . . 12

1 Specification and Design

1.1 General Architecture

The system is organized into three independent modules:

1. **Generation module** (`maze_generation.py`): produces mazes that are guaranteed to contain a solution.
2. **Solving module** (`dijkstra_solver.py`): computes the optimal path via optimized Dijkstra.
3. **Optimization module** (`genetic_algorithm.py`): proposes an evolutionary approach inspired by Darwinian theory.

Design Choices

Matrix representation. The maze is encoded as an $n \times n$ matrix where 0 is a wall and 1 is a path. This structure simplifies local operations (8-directional neighborhood), at the cost of some memory inefficiency for very large mazes.

Direction system. The 8 cardinal and diagonal directions are defined in a global list:

```
DIRECTIONS = [(0,1), (-1,1), (-1,0), (-1,-1),
              (0,-1), (1,-1), (1,0), (1,1)]
```

No algorithmic dependencies. Loops use only `while` and native Python structures (lists, dictionaries). Matplotlib and NumPy are used solely for visualization.

1.2 Maze Generation (DFS)

Algorithm: iterative depth-first search with backtracking.

The maze is initialized as a full grid of walls. A starting cell is marked as a path, then the algorithm recursively explores neighbors. A neighbor is *eligible* if:

- it has never been visited (value $= 0$);
- it has exactly one visited neighbor (the cell we came from).

This constraint guarantees that the path graph is acyclic and connected.

Termination. The stack contains only cells from the finite domain $n \times n$. Each cell is marked at most once; the stack strictly grows then shrinks, guaranteeing termination in finite time.

Complexity.

Step	Complexity
Initialization	$\mathcal{O}(n^2)$
Main loop (each cell pushed/popped exactly once)	$\mathcal{O}(n^2)$
Neighbor check (8×8 operations max per cell)	$\mathcal{O}(1)$ per cell
Total	$\mathcal{O}(n^2)$

1.3 Solving by Dijkstra (BFS)

Algorithm: the implementation uses BFS (*Breadth-First Search*) with a FIFO queue. This is valid because all edge weights are equal to 1.

The algorithm starts from the goal and propagates distances to all reachable path cells. For each visited cell, the distance to unvisited neighbors is incremented by 1.

Directional map. After computing distances, a second pass builds a directional map: each cell points to its neighbor with the smallest distance to the goal. Solving follows these pointers.

Termination. Each cell is enqueued once and dequeued once. The queue strictly decreases, guaranteeing termination.

Complexity.

Step	Complexity
Map initialization	$\mathcal{O}(n^2)$
BFS (each cell visited once, 8 edges per cell)	$\mathcal{O}(n^2)$
Directional map construction	$\mathcal{O}(n^2)$
Path following	$\mathcal{O}(n)$ on average
Total	$\mathcal{O}(n^2)$

1.4 Genetic Algorithm with Pheromones

Representation. An individual is a program: a list of genes, each gene being an integer in $\{0, \ldots, 7\}$ representing a movement direction.

Fitness function.

$$f = d_{\text{euclid}} + \lambda_1 \cdot \mathbf{1}[\text{collision}] + \lambda_2 \sum_{\text{pos} \in \text{path}} \text{pheromones}[\text{pos}] \tag{1}$$

where:

- d_{euclid} = Euclidean distance from the final position to the goal;

- $\lambda_1 = 0.1$: very low penalty per collision;
- $\lambda_2 = 2.0$: pheromone weight;
- $\mathbf{1}[\text{collision}]$ is an indicator equal to 1 if a collision occurred.

Selection. Strict elitist selection: the $\lfloor t_s \cdot N \rfloor$ best individuals (at least 2) are preserved intact.

Crossover. Random cut point in $[L/3,\ 2L/3]$, then combination of both parents. This avoids concentrating modifications at the extremities of the program.

Mutation. Each gene mutates with probability t_m. At least one mutation is forced if none occurred naturally and $r < t_m$.

Pheromones. After each generation, poor individuals (fitness > average) deposit a pheromone at their final position. Pheromones evaporate by 5% per generation to avoid blocking future exploration.

Termination. Stop if the goal is reached or after n_G generations.

2 Code Organization

2.1 Architecture

```
src/
  maze_generation.py
    create_empty_matrix()
    count_visited_neighbors()
    get_eligible_neighbors()
    generate_maze()
    display_maze()
    Tests

  dijkstra_solver.py
    initialize_map()
    dijkstra()
    build_directional_map()
    solve_maze()
    visualize_distances()
    visualize_solution()
    Tests

  genetic_algorithm.py
```

```
    create_random_individual()
    initialize_population()
    execute_program()
    calculate_fitness()
    select_best()
    crossover()
    mutation()
    genetic_algorithm()
    Tests

  main.py
    plot_exploration()
    plot_solution()
    plot_fitness()
    main()
```

2.2 Control Flow

Initialization. `main()` → `generate_maze(N)` → `create_empty_matrix()` → iterative DFS with stack.

Solving. `dijkstra(maze, goal)` → `initialize_map()` → BFS → `build_directional_map()` → `solve_maze()`.

Stochastic optimization. `genetic_algorithm(...)` → `initialize_population()` → generation loop:

1. `execute_program()` for each individual
2. `calculate_fitness()`
3. `select_best()`
4. `crossover()` + `mutation()`
5. pheromone update

(stop if goal reached or n_G exceeded)

Visualization. `plot_*()` → NumPy → Matplotlib → PNG export.

2.3 Memory Management

- **DFS / Dijkstra:** $\mathcal{O}(n^2)$ matrices. Acceptable for $n \leq 500$.
- **GA:** population of $N = 2000$ individuals $\times$ $L = 1200$ genes $\approx$ 2.4M integers.
- **Pheromones:** n^2 float matrix recomputed each generation.

3 Installation and Usage

3.1 Requirements

Python 3.8 or higher.

```
pip install numpy matplotlib
```

3.2 Running

From the `src/` directory:

```
# Full pipeline
python main.py

# Individual modules
python maze_generation.py
python dijkstra_solver.py
python genetic_algorithm.py
```

Results are saved to `results/final/`:

- `1_Dijkstra_Exploration.png` — distance heatmap
- `2_Dijkstra_Solution.png` — optimal path
- `3_GA_Fitness.png` — convergence curve
- `4_GA_Exploration.png` — GA exploration heatmap
- `5_GA_Solution.png` — best GA path

3.3 Configurable Parameters

In `main.py`:

```
MAZE_SIZE = 100    # N
N_POP     = 2000   # GA population size
L_PROG    = 1200   # Max program length (genes)
TS        = 0.1    # Selection rate (10%)
TM        = 0.3    # Mutation rate
NG        = 1000   # Number of generations
```

4 Behavioral Analysis

4.1 Empirical Complexity

Dijkstra on $n \times n$ mazes (100 measurements).

N	Time (s)	Expected $\mathcal{O}(n^2)$
5	0.0001	~ 25
50	0.0015	$\sim 2{,}500$
100	0.006	$\sim 10{,}000$
500	0.15	$\sim 250{,}000$

Confirmed: practical complexity $= \mathcal{O}(n^2)$ with very low constant overhead.

Genetic Algorithm (500 generations).

N	Time (s)
50	2.5
100	12
500	180

Non-linear: $N = 500$ is $72\times$ more expensive than $N = 50$.

4.2 Fitness Convergence (GA)

Population 2000, 1000 generations on a 100×100 maze:

Generation	Best score	Avg. score	Note
0	85,000	102,000	Random initial population
10	45,000	68,000	−47%
50	15,000	28,000	−82%
100	8,000	12,000	−91%
500	3,200	5,100	−96%
1000	2,800	4,900	−97%

Observed behavior:

- **Fast exploration phase** (gen. 0–50): near-exponential improvement.
- **Exploitation phase** (gen. 50–500): local refinement.
- **Plateau** (gen. 500+): saturation, marginal further improvement.

Fitness = distance + penalties; a lower score means the individual is closer to the goal.

4.3 Path Lengths

Dijkstra (optimal).

- $N = 50$: average length 42
- $N = 100$: average length 78
- $N = 500$: average length 320

GA after 1000 generations.

- $N = 50$: 45–50 (very close to optimal)
- $N = 100$: 88–95 (approx. 10% above optimal)
- $N = 500$: 380–420 (approx. 30% above optimal)

The GA produces viable paths quickly at small scale, but quality degrades progressively on larger mazes.

4.4 GA Parameter Sensitivity

Effect of selection rate t_s ($N = 100$, 500 generations).

t_s	Conv. gen.	Final path
0.05	350	92
0.1	280	86
0.2	320	89
0.3	200	85

Optimal $\approx$ 0.1–0.3: low t_s means less selection pressure and more exploration; high t_s means elitist pressure and faster convergence.

Effect of mutation rate t_m.

t_m	Diversity	Convergence
0.05	Low	Fast $\rightarrow$ premature convergence
0.1	Good	Stable
0.3	High	Slow (noise-dominated)

Optimal $\approx$ 0.1–0.2.

Effect of program length L.

L	Flexibility	Time/gen.	Final quality
100	Low	0.01 s	Poor (insufficient)
400	Medium	0.04 s	Good
1200	High	0.12 s	Very good

L too short makes reaching a distant goal impossible; L too long dilutes mutation effectiveness.

4.5 Exploration vs. Exploitation of Pheromones

With pheromones:

- First half (gen. 0–500): aggressive exploration; pheromones steer individuals away from dead-end positions.
- Second half (gen. 500–1000): convergence toward the best found individual.

Without pheromones: more uniform exploration pattern, less emergent structure.

Estimated quantitative gain: +15 to +20% convergence speed with pheromones enabled.

4.6 Dijkstra vs. GA Comparison

Criterion	Dijkstra	GA
Optimality	100% (optimal)	85–95%
Time $N = 100$	0.006 s	12 s
Time $N = 500$	0.15 s	180 s
Complexity	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2 \times \text{pop})$
Flexibility	Fixed	Adaptable (pheromones, fitness)
Robustness	Proven	Not guaranteed

Conclusion: Dijkstra is preferred for guaranteed optimality and speed. The GA is relevant for experimentation and observing emergent behaviors.

4.7 Connectivity

DFS systematically generates connected mazes:

- Reachable cells (Dijkstra) / total: 100% (by construction).
- No isolated points observed in any test run.

5 Formal Proofs

Theorem 5.1 (Termination — DFS)**.** *The DFS algorithm terminates in finite time for any size n.*

Proof. The stack contains only cells from the finite domain $n \times n$. At each iteration, either a new eligible neighbor is discovered and pushed, or no eligible neighbor exists and the current cell is popped. The stack can only grow then strictly decrease; it therefore empties in finite time. □

Theorem 5.2 (Correctness — DFS)**.** *The DFS algorithm produces a connected maze.*

Proof. The produced graph is acyclic: each path cell has exactly one predecessor, except the root. Each discovery adds an edge between two existing components. By induction on the number of steps, the graph remains connected at every step. □

Theorem 5.3 (Optimality — Dijkstra/BFS)**.** *The path produced by BFS has minimal length.*

Proof. By induction on distance k.

Base case: the cell at distance 0 (the goal) is trivially optimal.

Inductive hypothesis: every cell at distance $d \leq k$ has an optimal path from the goal.

Inductive step: let C be a cell at distance $k+1$. BFS reaches it from a neighbor V at distance k (optimal by hypothesis); the resulting path has length $k+1$. Suppose by contradiction that a shorter path to C of length $\leq k$ exists. That path must traverse a cell at distance $\leq k$, but BFS would then have visited C earlier — contradiction.

Therefore BFS produces a path of minimal length. □

5.1 GA Convergence

Theorem 5.4 (Convergence — GA (theoretical))**.** *A genetic algorithm converges to an optimal solution with probability 1, under sufficient exploration conditions and with an infinite population.*

This theorem does not apply to a finite practical implementation. Empirical observations yield the following:

- If $L <$ optimal path length, reaching the goal is impossible regardless of other parameters.
- The population can stagnate in suboptimal regions.
- After n generations, elitist selection reduces genetic diversity, which limits exploration and causes stagnation.

Attempted improvements and their limitations:

- Penalizing poor paths with pheromones: tends to block exploration if the weight is too high.
- Increasing L: program too long, mutations become proportionally less effective.

6 Implemented Optimizations

Dijkstra. Replacing the priority queue ($\mathcal{O}(n^2 \log n)$) with a `deque`-based BFS ($\mathcal{O}(n^2)$), valid because all edge weights are unitary.

Collision check. $\mathcal{O}(1)$ via direct array access: `maze[i][j] == 0`.

Pheromones. Without pheromones, the GA explores 8^L potential solutions approximately uniformly. With pheromones, the search concentrates around promising paths.

Strict elitism. The best individual found so far is always preserved in the next generation, preventing fitness regression.

7 Experimental Results

7.1 Mazes Tested

Size	Generation time
5×5	< 0.001 s
50×50	0.001 s
100×100	0.005 s
500×500	0.08 s

7.2 Dijkstra Solutions

N	Path length	Solving time
5	5	< 0.001 s
50	42	0.001 s
100	78	0.005 s
500	320	0.15 s

7.3 GA Solutions (1000 generations)

N	Length (with pheromones)	Conv. gen.
50	45	200
100	85	350
500	390	> 1000

The GA performs well on small N; quality degrades progressively as n increases.

8 Conclusion

8.1 Key Results

1. DFS generates non-trivial connected mazes in $\mathcal{O}(n^2)$.
2. Optimized Dijkstra (BFS) solves mazes in $\mathcal{O}(n^2)$, outperforming the classical priority-queue implementation.
3. The GA with pheromones produces solutions of variable quality, but:

- explores the search space creatively;
- is adaptable to various fitness variants.

4. Dijkstra guarantees optimality and is near-instantaneous; the GA is flexible but slow and without convergence guarantees.

8.2 Possible Improvements

- Implement A^* for a heuristically guided search.
- Parallelize fitness evaluation (threads or multiprocessing).
- Enrich the fitness function: distance + hazard zones + domain constraints.
- Post-optimize found paths (shortcut removal: $A \to B \to C$ replaced by $A \to C$ when feasible).
- Deposit pheromones on *good* paths rather than poor ones.
- Extend to dynamic mazes with moving obstacles.
- Validate on real-world structures (building floor plans, road networks).

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.

[2] J. H. Holland. *Adaptation in Natural and Artificial Systems*. MIT Press, 1992.

[3] M. Dorigo, T. Stutzle. *Ant Colony Optimization*. MIT Press, 2004.